

Python basics refresh and OOP (L02-L03)

Python syntax, data types & standard libraries

Full knowledge Python knowledge and compliance?



Guido Van Rossum

Google ✅

the web totality

In this course we take for granted basis of python syntax, data types and standard libraries.

BUT every programmer never stops searching online, programming is an iterative process that is always learnt by doing!

Useful comprehensive cheatsheet

Script execution

To run a python script there are different ways. The base one is by executing the following command:

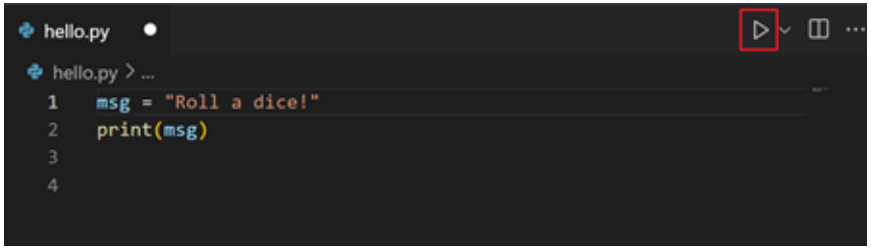
- *python script.py*
  - interpreter
  - script path

What happens?

1. Python read the source file (.py)
2. parse it to check syntax → Syntax errors
3. compiles it into bytecode (intermediate format)
4. executes it line by line → Execution errors

VS code

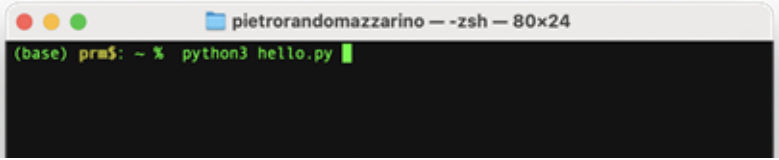
vs code documentation



=

From terminal





Key concepts

Everything in Python is an OBJECT!
Objects are characterized by *type* , *value* and *memory id*. When you assign a variable name to an object what happens is like labelling a specific memory id with a name.

x = 5

python stores the *integer* object with *value five* in a specific *memory slot* and makes the name ‘x’ point to it x —> 5

Immutable objects

int, float, str, tuple

Mutable objects

list, dict , set

```
# Immutable
a = 5
b = a
a = a + 1
print(b)  # 5 (unchanged)

# Mutable
list1 = [1, 2, 3]
list2 = list1
list2.append(4)
print(list1)  # [1, 2, 3, 4]
```

Key concepts

INDENTATION is FUNDAMENTAL! → this is how the interpreter knows what goes with what creating logical blocks. NB either 1 tab or 4 spaces, stay consistent.

Indentation is required for different scopes:

Inside `if` → runs conditionally

Inside `for` or `while` → runs repeatedly

Inside `def` → defines function body

Inside `class` → defines class body

```
for i in range(3):
    print(i)
print("Loop finished")  # runs once, after the loop


x=10
if x > 0:
print("Positive") #IndentationError


x=10
if x > 0:
    print("Positive")


for i in range(3):
    msg = "Hello"
print(msg)  # ❌ NameError, msg not visible outside the function



for i in range(3):
    msg = "Hello"
    print(msg)  # ✅ No error, msg is visible within the function
```

Key concepts

Function definition and usage.

A Function takes arguments and return something when return is omitted it automatically returns None.

arguments can be : positional, keywords matched, default, variable length positional variable length keyword matched

Exercise (L01, already done):

celsius_to_fahrenheit, is_too_cold, is_too_hot, hvac_command

File:
2026_2027/L01_intro/solution/ex1_functions.py



Object Oriented Programming

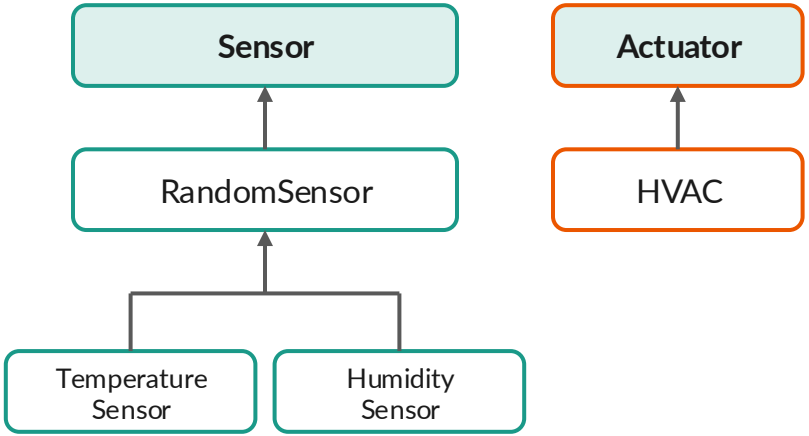
Base knowledge of OOP in Python

Core OOP concepts

1. **Class** – A blueprint for creating objects.
2. **Object** – An instance of a class.
3. **Attributes** – Variables that store data.
4. **Methods** – Functions that belong to a class.
5. **Inheritance** – Reuse and extend existing classes.
6. **Encapsulation** – Hide internal details.
7. **Polymorphism** – Use same interface for different types (method overriding)

Our domain: a smart-home room

- A room has a temperature sensor, a humidity sensor and an ON/OFF HVAC.
- No physics: sensors return random values (or CSV rows); the HVAC does not change the measurements.
- The classes we write are the ones of the sample final project: smart_home/device_connector/devices.py.



Arrows point to the parent class (inheritance, "is-a").

Exercise 1 - base classes

Goal: define the base classes every device of the project inherits from.

Task: Sensor(name, unit) stores name and unit; sense() raises NotImplementedError. Actuator(name) stores name; actuate(command) raises NotImplementedError.

Check: Sensor("x", "u").sense() fails with NotImplementedError.

File: L02_oop_basics/solution/devices.py

```
class Sensor:
    def __init__(self, name, unit):
        self.name = name
        self.unit = unit

    def sense(self):
        raise NotImplementedError
```

Exercise 2 - concrete devices

- Goal: specialise the base classes (inheritance) and use them in the same way (polymorphism).
- Task: RandomSensor(Sensor) with low, high: sense() returns round(random.uniform(low, high), 1). TemperatureSensor (15-28 Cel) and HumiditySensor (40-80 %RH) are RandomSensors.
- HVAC(Actuator): state starts "OFF"; actuate("ON" or "OFF"), anything else raises ValueError.
- File: L02_oop_basics/solution/devices.py

Polymorphism: same call, different sensors

```
for s in [TemperatureSensor(), HumiditySensor()]:
    print(s.name, s.sense(), s.unit)
```

Exercise 3 - command line

Goal: use the classes from a small command-line program.

Task: a menu loop with input() (menu on the right): read the sensors, HVAC ON, HVAC OFF, quit. Any other input: "invalid choice", then the menu again.

File: L02_oop_basics/solution/main.py (run: python main.py)

```
1) read sensors
2) HVAC ON
3) HVAC OFF
q) quit
```

Exercise 4 (extra) - sensor from a CSV file

Goal (extra): a sensor that replays real data, used exactly like the random ones.

Task: CsvSensor(Sensor) reads one column (DryBulb °C or RelHum %) of 2026_2027/data/meteo_frassinetto.csv; each sense() returns the next row and restarts at the end.

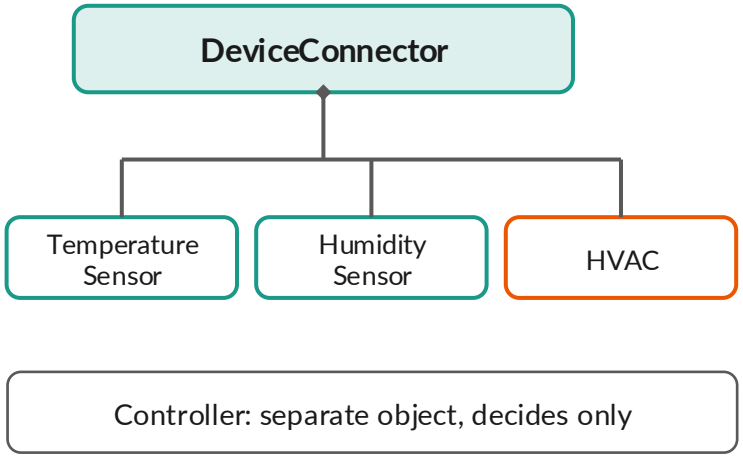
Check: print 5 readings of each; the first temperature must be -1.68.

Files: L02_oop_basics/solution/devices.py, csv_demo.py

```
self.values = pd.read_csv(path)[column].tolist()
```

L03 - Inheritance vs composition

- Inheritance = "is-a": TemperatureSensor is a Sensor.
- Composition = "has-a": a DeviceConnector has sensors and an HVAC, passed in the constructor.
- Separation of duties: the connector reads and acts, the controller decides, main passes data.
- Config instead of code: adding a room = editing config.json (exam requirement).



Diamond = "has-a" (composition)

Exercise 5 - DeviceConnector

Goal: one object per room: it HAS sensors and an HVAC (composition).

Task: DeviceConnector(house_id, room_id, sensors, hvac). read_sensors()

→ {"temperature": 21.4, "humidity": 55.0}, stored in self.latest; actuate(command) forwards it to the HVAC; status() → house, room, latest, HVAC state. No control logic here.

File: L03_oop_composition/solution/device_connector.py

```
DeviceConnector(house_id, room_id, sensors, hvac)
```

Exercise 6 - Controller

- Goal: keep the decision out of the device: the controller only decides.
- Task: Controller(thresholds).decide(measurements) returns (command, alert):
  - temperature < t_min → "ON"; > t_max → "OFF"; inside the band → None. humidity > h_max → alert such as "humidity 78% > 70%", otherwise None.
- File: L03_oop_composition/solution/controller.py

```
if m["temperature"] < t["t_min"]:
    command = "ON"
elif m["temperature"] > t["t_max"]:
    command = "OFF"
```

Exercise 7 - rooms from config.json

- Goal: build the system from configuration, not from code (exam requirement).
- Task: build_rooms() reads config.json and builds one DeviceConnector (random sensors + HVAC) and one Controller per room. Loop: read, decide, actuate only if the command differs from the HVAC state. Then add a room to config.json and run again, without touching the code.
- File: L03_oop_composition/solution/main.py

```
cd L03_oop_composition/solution
python main.py 5        # 5 cycles, 1 s apart
```